Tabular Q-learning vs Deep Q-Networks

Sample efficiency, sparse rewards and DQN ablations

Aispur Santiago · Amaya Bryan

Intelligent Agents · Instructor: PhD Armas Rolando · November 2026

10 seeds per configuration · 170 runs · 22.3 CPU-hours

The question and the rules we followed

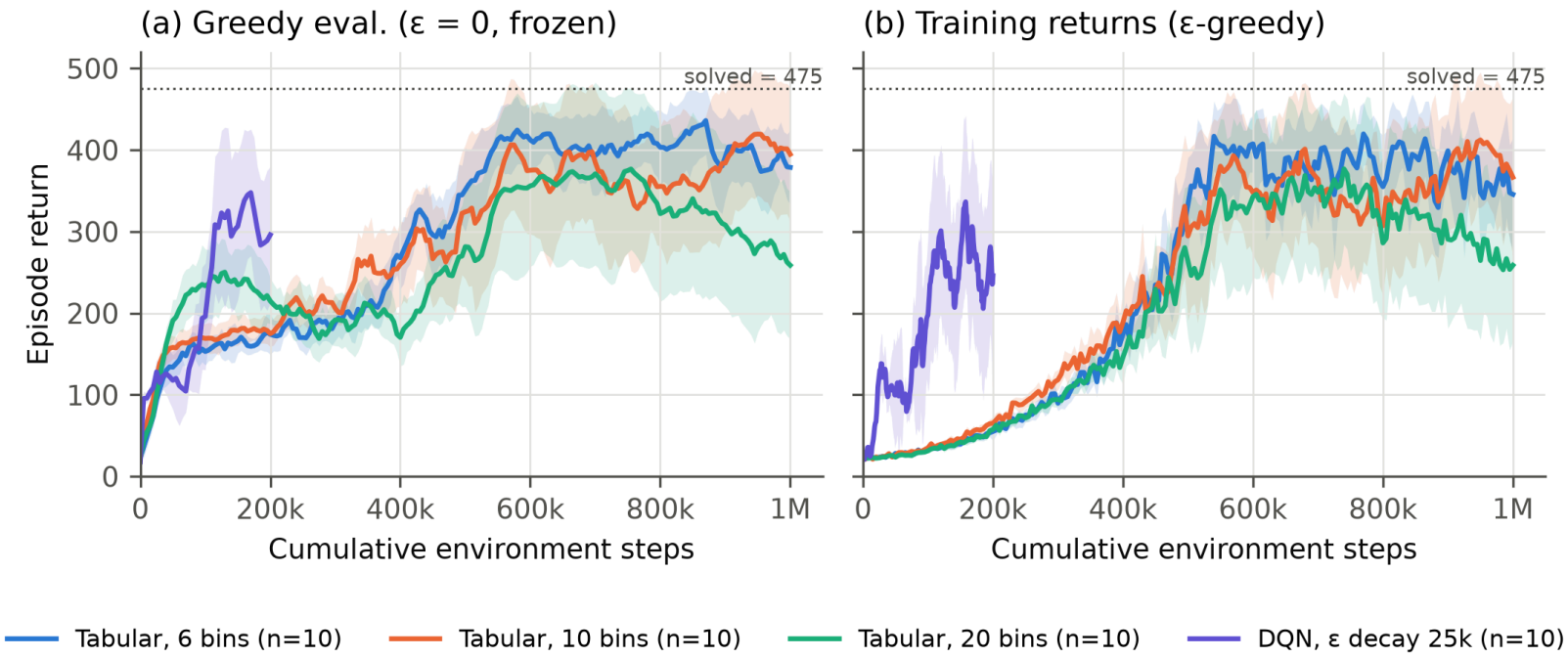
Question: not only *whether* an agent learns, but *why* it wins or loses under a fair protocol.

1. **Common clock:** cumulative **environment steps**, never episodes
2. **Greedy evaluation** ($\varepsilon = 0$, frozen weights) every 5k/10k steps, shown apart from training returns
3. **10 fixed seeds**, 95% Student-t bands ($t_9 = 2.26$), Welch tests for key claims
4. **Negative results** are reported and diagnosed
5. **One command** regenerates everything

Studies

- **Tier 1** CartPole-v1: tabular (6/10/20 bins, angle edges) vs DQN (ε-decay 5k/25k/100k)
- **Tier 1** MountainCar-v0: sparse reward, planned negative result
- **Tier 2** LunarLander-v3: no target net, no replay, Double, Dueling
- **Theory:** limits of both families

CartPole: DQN needs far fewer samples



Greedy evaluation (left) and training returns (right); bands = 95% CI over 10 seeds.

56k steps (DQN) vs **382k** (best table) to reach return 475: **6.9× more sample-efficient**

…but each DQN step costs 23× more compute

analysis.py — CartPole sample efficiency vs compute

```
$ python analysis.py   # excerpt of its output
### 1.2 Sample efficiency vs compute (tabular 10 bins vs best DQN)
- steps to first greedy >= 475: tabular 382k vs DQN 56k -> DQN is 6.9x more SAMPLE-efficient
- CPU per env step: tabular 0.097 ms vs DQN 2.243 ms -> 23x
- CPU-seconds to reach 475: tabular 37 s vs DQN 124 s -> tabular is 3.4x more COMPUTE-efficient
- greedy return at an EQUAL budget of 200k steps (mean of checkpoints 180k-200k): tabular_bins6 174.1 ± 24.7 (n=10);
    tabular_bins10 174.9 ± 16.3 (n=10); tabular_bins20 214.9 ± 27.8 (n=10); dqn_eps5k 355.6 ± 95.4 (n=10); dqn_eps25k
    296.1 ± 83.5 (n=10)
- Welch t-test DQN(eps5k) vs tabular(10 bins) at 200k: t = 4.22, p = 0.002

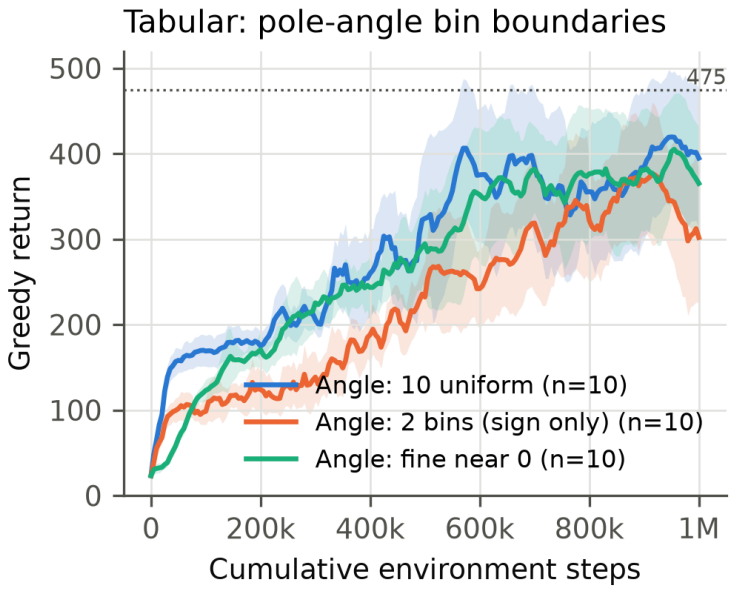

### 1.3 Number of bins: precision vs data
- tabular_bins6: 1,296 states, visited 38.7% (~502 states); final 355.8 ± 51.0 (n=10); seed-to-seed std of final = 71
- tabular_bins10: 10,000 states, visited 18.0% (~1,801 states); final 392.2 ± 81.4 (n=10); seed-to-seed std of final = 114
- tabular_bins20: 160,000 states, visited 4.6% (~7,382 states); final 263.0 ± 95.8 (n=10); seed-to-seed std of final = 134
- Welch t-test final(6 bins) vs final(20 bins): t = 1.93, p = 0.074

[exit 0 | wall time: 1.3 s  (whole script, 170 cached runs)]
```

Output of `python analysis.py` (1.3 s on the cached runs).

Reaching 475 costs **37 CPU-s** (table) vs **124 CPU-s** (DQN): the table is **3.4× cheaper**.

Bins and pole-angle boundaries

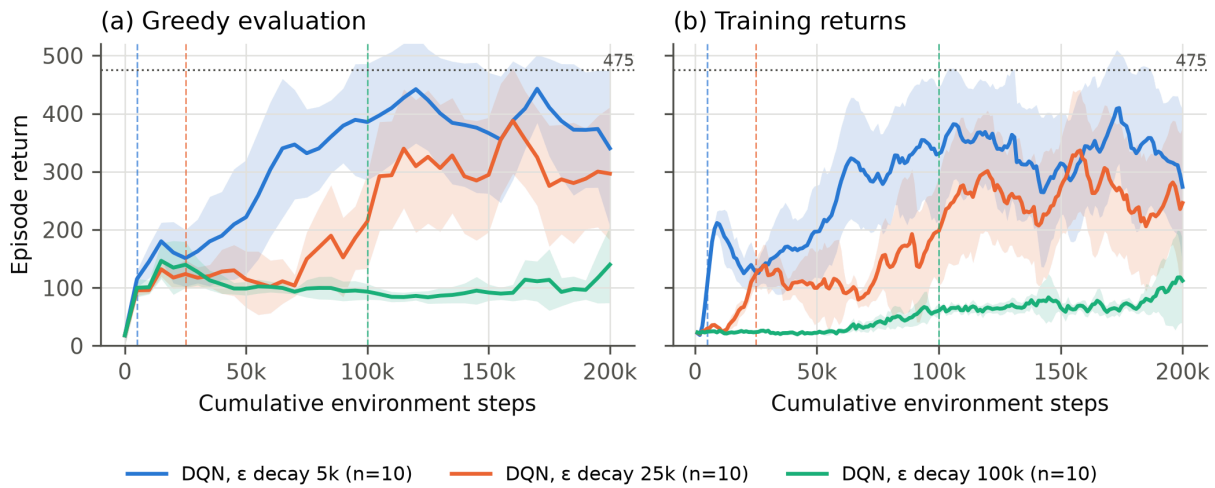


Angle study: sign-only angle is slower early ($p < 10^{-6}$)

Bins	Cells	Visited	Final
6	1,296	38.7%	356 ± 51
10	10,000	18.0%	**392 ± 81**
20	160,000	4.6%	263 ± 96

- Too coarse: cannot express a precise policy
- Too fine: data-starved, 95% of cells never updated
- Resolution must be paid for with data

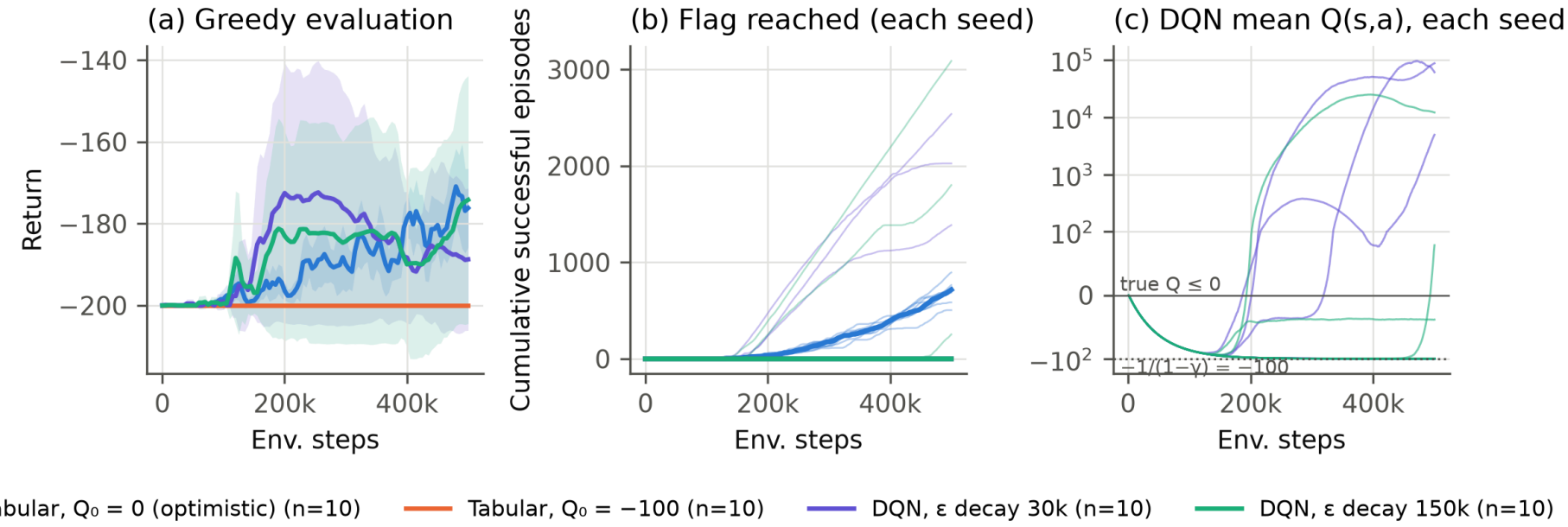
Catastrophic forgetting and ε-decay



- **20/20** fast- and medium-decay DQN seeds reached ≥ 475 and later fell below 200
- Judged only at the last checkpoint, a solved agent looks mediocre → **evaluate during training**
- Fast decay wins on CartPole (random actions already cover the useful states); 100k decay: only 2/10 seeds solve it

Cause: a good policy fills the buffer with near-upright states; the network overwrites what it knew about large tilts.

MountainCar: DQN fails, and we can say why



(a) greedy return, (b) seeds that found the flag, (c) mean Q. Before the first flag every DQN sits at Q = −100.

Optimistic table ($Q_0 = 0$)
flag found **10/10**

Pessimistic table ($Q_0 = -100$)
flag found **0/10**

DQN
flag found **6/20**; 14 runs stuck at Q ≈ −99.4

MountainCar: when DQN finds the flag, Q diverges

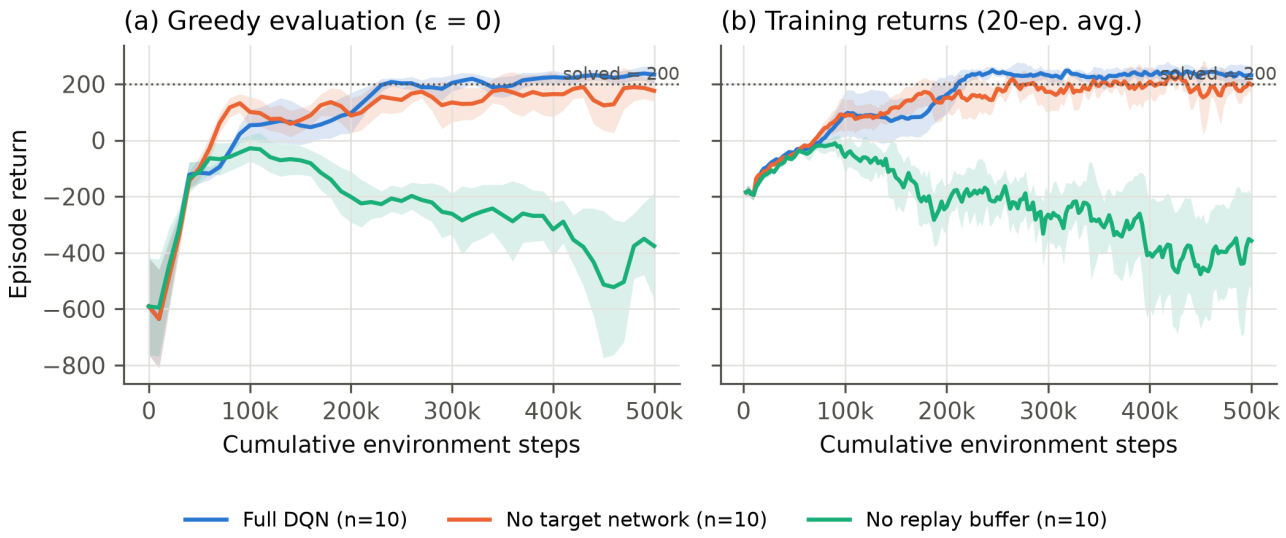
analysis.py — MountainCar: Q-divergence check

```
$ python analysis.py   # excerpt of its output
- reference: if the flag is never seen, every Q converges to -1/(1-gamma) = -100
- DQN runs that NEVER reached the flag: n=14, final mean Q -99.4 ± 0.1 (n=14), final greedy -200.0 ± 0.0 (n=14)
- DQN runs that DID reach it: n=6, final mean Q 26751.0 ± 38494.7 (n=6), final greedy -138.2 ± 37.5 (n=6)
- Q divergence check (all true values are <= 0 in MountainCar):
  - dqn seed 2: first flag at 87k; mean Q before -51, max after 4923; grad-norm before 0.4, max after 658
  - dqn seed 5: first flag at 94k; mean Q before -54, max after 87233; grad-norm before 0.4, max after 7861
  - dqn seed 8: first flag at 94k; mean Q before -54, max after 95715; grad-norm before 0.4, max after 5322
  - dqn_long_eps seed 2: first flag at 107k; mean Q before -58, max after 24799; grad-norm before 0.4, max after 1881
  - dqn_long_eps seed 4: first flag at 416k; mean Q before -87, max after 79; grad-norm before 0.4, max after 84
[exit 0 | wall time: 1.3 s]
```

Excerpt of `python analysis.py`: true values are ≤ 0, yet mean Q reaches +95,715.

- Value cliff at the flag + `max` in the bootstrap target → the **deadly triad**. A unit test confirms terminal targets are y = r (not a done-flag bug).

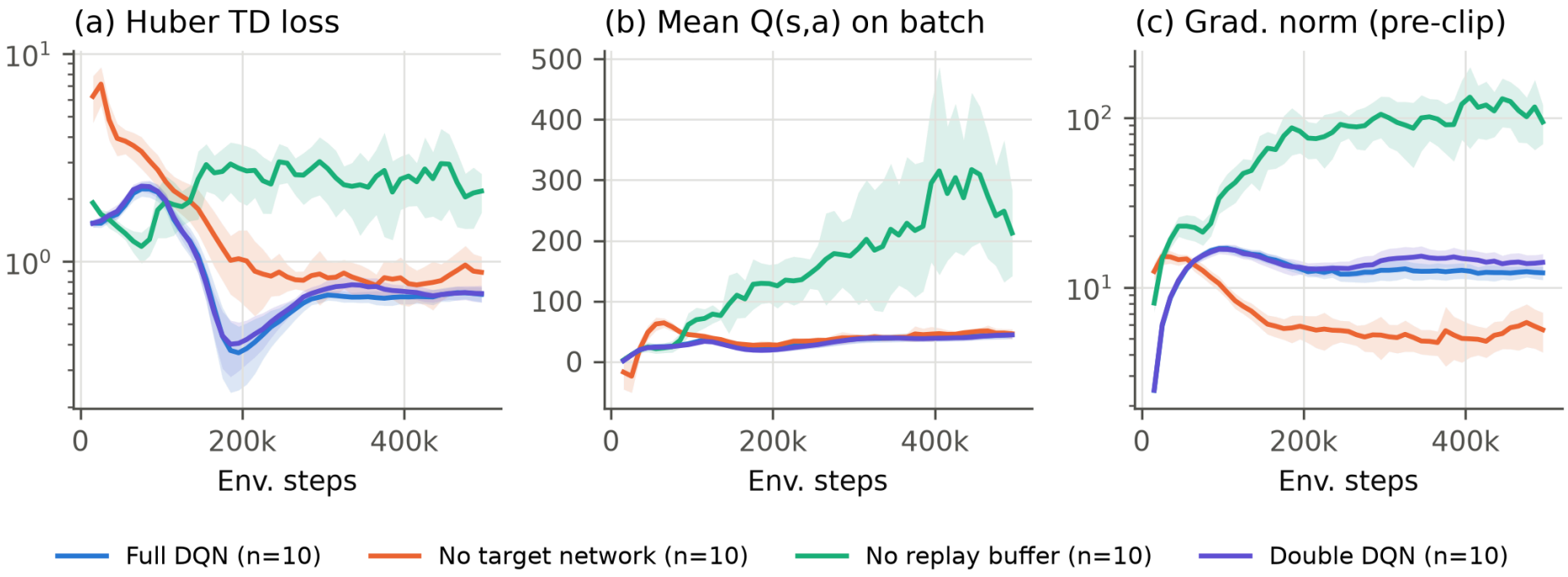
LunarLander ablations: what each DQN component buys



Config	Final	p vs full
Full DQN	234 ± 29	—
No target net	176 ± 37	0.014
No replay	−376 ± 188	4e−5
Double	234 ± 29	0.98
Dueling	238 ± 33	0.82

- **Target net = stability:** same peak (275), but 10/10 seeds later crash below 0
- **Replay = learning at all:** peak 149, ends at −376
- **Double/Dueling:** null result, reported as such

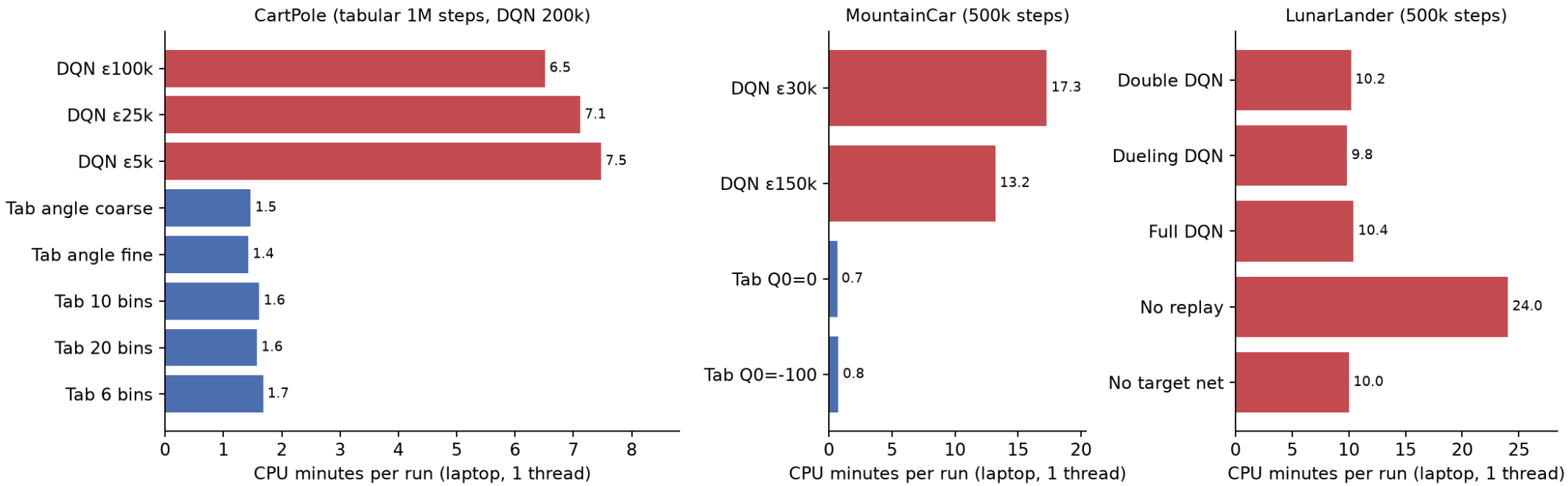
Diagnostics explain the collapse



TD loss, mean Q and pre-clip gradient norm.

No replay: mean Q **5.9×** the full agent’s (256.9 vs 43.4) while the policy is far worse; gradients 300 vs 20.

Time taken (1): the 170 experiment runs



CPU minutes per run on the reference laptop (Ryzen 5 7235HS, 1 thread). Blue = tabular, red = DQN.

1M-step table ≈ 1.5 min
200k-step DQN ≈ 7 min

MountainCar table 0.7 min
vs DQN 13–17 min

Total: 170 runs = **22.3 CPU-hours** (3 h on 8 threads)

Time taken (2): fresh runs measured today

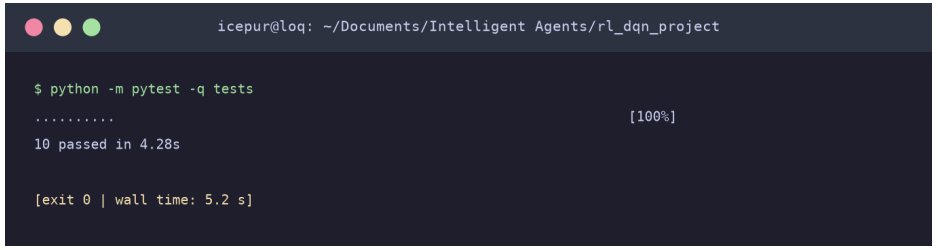
Fresh (uncached) runs — timed today

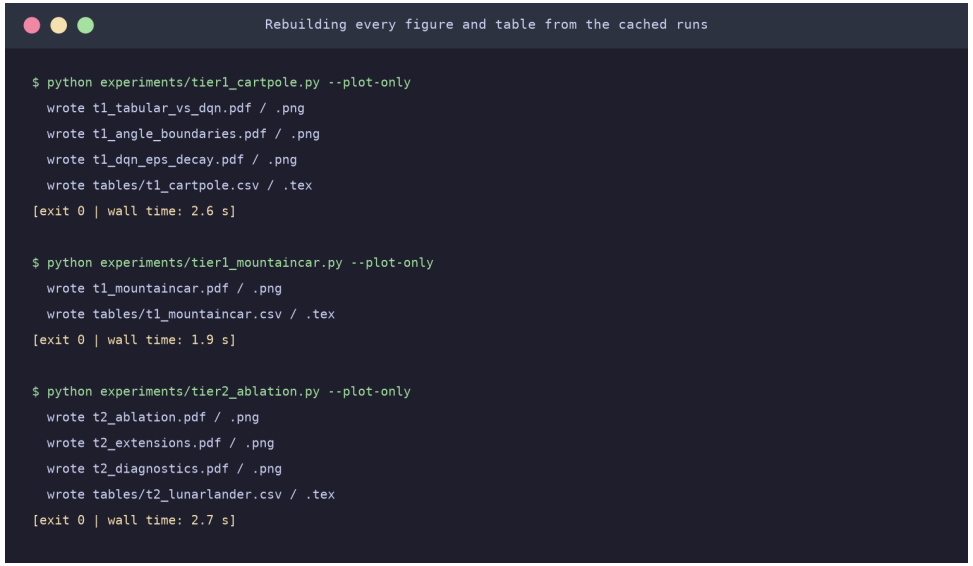
```
Live timing on this laptop (Ryzen 5 7235HS), seed 0, fresh runs
Tabular Q-learning, 10 bins: 200,000 steps | wall 11.0s | cpu 10.9s | 0.055 ms CPU/step
   greedy eval (every 5k): [15, 94, 90, 117, 172, 145, 164, 167, 165, 161, 170, 141, 154, 190, 156, 155, 161, 211, 131,
    188, 152, 219, 139, 218, 218, 171, 220, 220, 224, 161, 187, 220, 211, 220, 224, 130, 171, 221, 228, 264, 264]
DQN (eps decay 5k)         : 30,000 steps | wall 33.1s | cpu 33.0s | 1.100 ms CPU/step
   greedy eval (every 5k): [93, 169, 162, 101, 104, 101, 108]
```

New uncached runs, seed 0, same laptop, 4 Nov 2026.

Run	Wall time	Result
Tabular 10 bins, 200k steps	11.0 s	greedy eval 264
DQN (ε 5k), 30k steps	33.1 s	greedy eval 108 (early in training)
`pytest -q tests` (10 tests)	5.2 s	10 passed
`analysis.py`	1.3 s	all tables regenerated

Reproducibility: one command, figures in seconds






```
python experiments/run_all.py   # 170 runs
python analysis.py              # every number
python -m pytest -q tests       # 10 tests
cd report && latexmk -pdf main.tex
```

- Seeds: `random`, `numpy`, `torch`, agent RNG, `env.reset(seed=k)`; fixed evaluation seeds
- Every run is a JSON with full spec; cached runs are skipped
- Seeds can be split across machines (`--seed-start`)
- Same-seed runs match bit for bit (tested)

Why neither family scales: theory

Case	Size	Consequence
HalfCheetah, 6-D action × 5 levels	$5^6 = 15{,}625$ actions	DQN output layer and max over 15k actions; exploration hopeless
17-D state, 10 bins/dimension	10^{17} cells	Table too large, never visited
84×84 grayscale Atari frame	256^{7056} states	Exact revisits never happen
Our CartPole, 6→20 bins	1,296 → 160,000	Visited fraction falls 39% → 4.6% (measured)

Instead: PPO / SAC / TD3 for continuous actions; convolutional function approximation for pixels.

Conclusions

1. DQN wins on **samples** (7×); tabular wins on **compute** (3.4×): it depends on what is scarce
2. Resolution costs data: **10 bins** beat 6 and 20
3. **Evaluate during training**: DQN solves CartPole and then forgets
4. **Optimism is exploration**: $Q_0 = 0$ vs -100 flips 10/10 to 0/10
5. Sparse reward → flat Q = −100 → no gradient; and the deadly triad can make Q diverge
6. Target net buys **stability**, replay buys **learning**; Double/Dueling: no effect at this scale
7. Rigor changes conclusions: 5 → 10 seeds reversed our 6-vs-10-bins ranking